# Vorlesung: Grundlagen der Computergrafik

Kapitel 5:

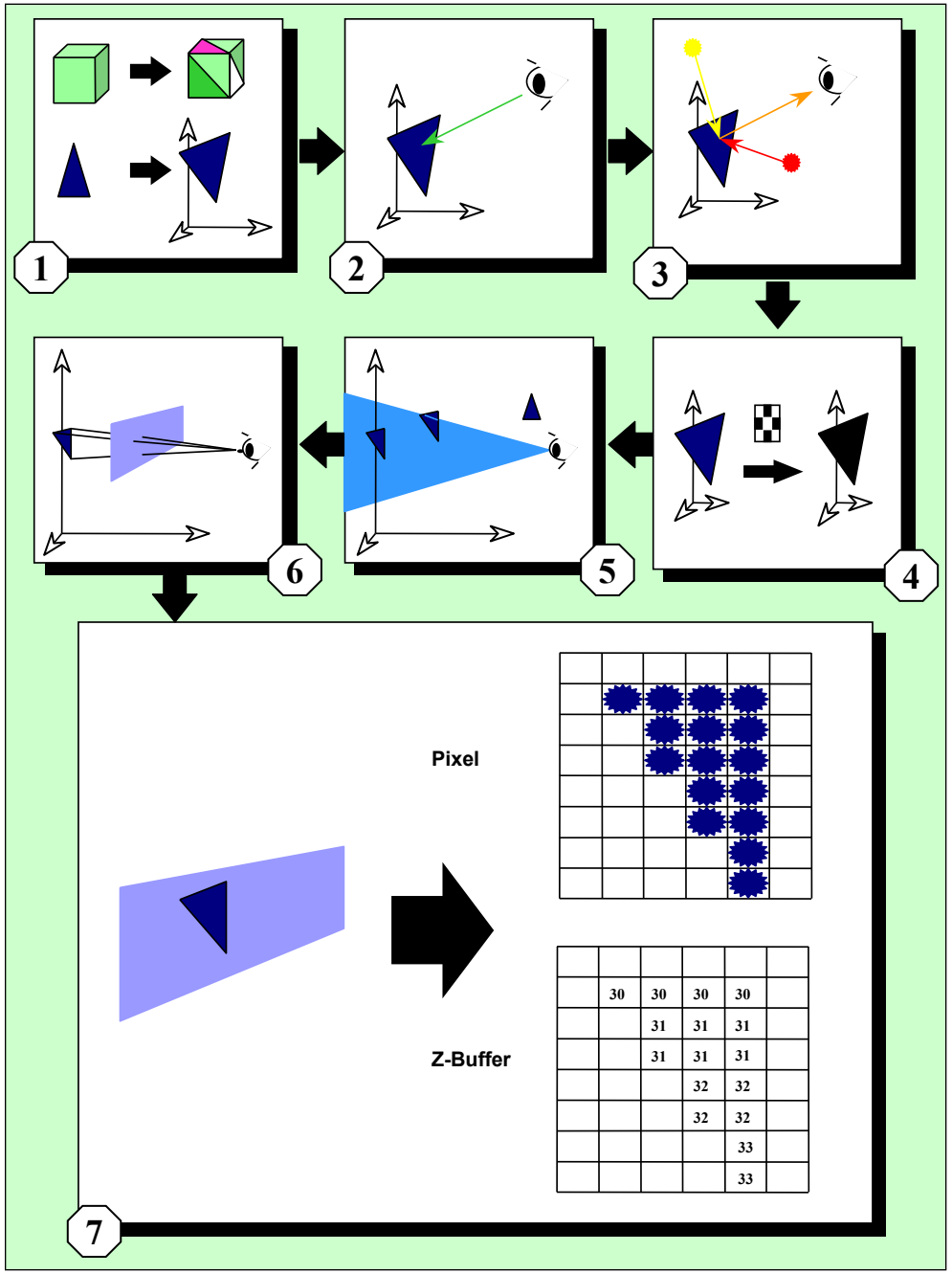
# Radiosity und Ray Tracing



Vorlesung im Sommersemester 2026
Prof. Dr. Stephan Diehl, Informatik, Universität Trier



# Klassische Rendering Pipeline



# Weiter zu

- Kapitel 5.1: Radiosity
- Kapitel 5.2: Ray Tracing

# Vorlesung: Grundlagen der Computergrafik

Kapitel 5: Radiosity und Ray Tracing



## Kapitel 5.1:

# Radiosity

**Vorlesung im Sommersemester 2026**

**Prof. Dr. Stephan Diehl, Informatik, Universität Trier**

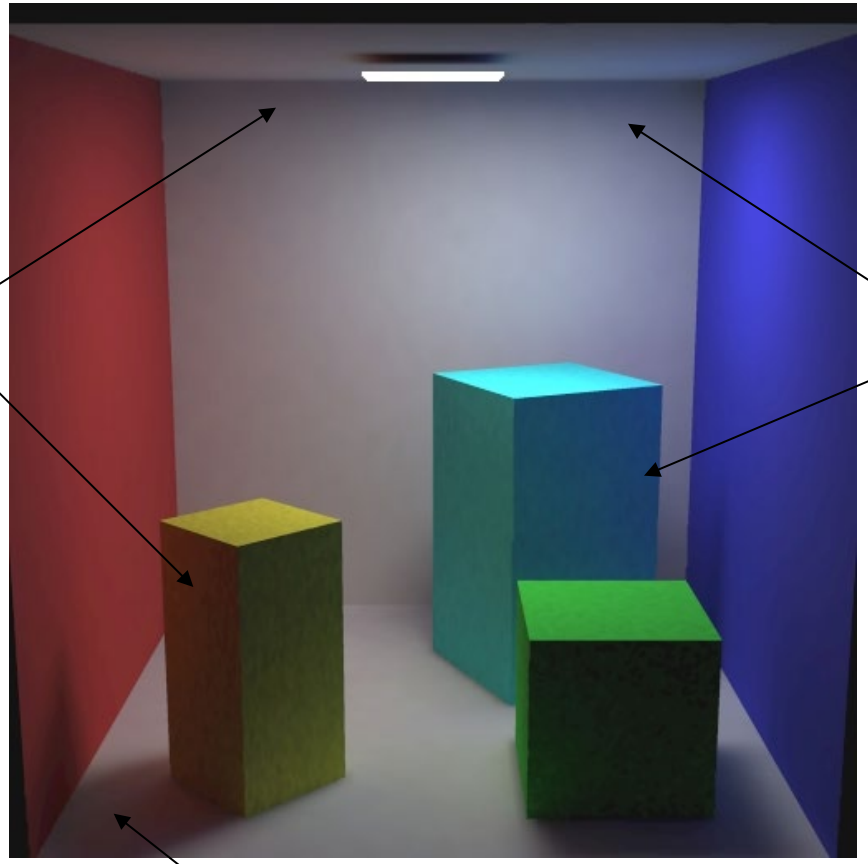
# Radiosity

- Radiosity (ca. 1984 eingeführt) geht von der **Erhaltung der Lichtenergie in einer geschlossenen Szene** aus. Jeder von einer Fläche abgestrahlte oder reflektierte Energie entspricht die Reflexion oder Absorption durch andere Flächen.

- **Energiebilanz** der Facetten wird aufgestellt (wie viel Licht wird empfangen, wie viel wohin ausgestrahlt).
  - **nicht gut bei Spiegelungen und Brechungen**
  - **unabhängig vom Beobachterstandpunkt (gut für Animation),**
  - **sehr gut für diffus reflektiertes Licht (Innenraum Szenen)**

- **Die gesamte von einer Fläche abgegebene Energie heißt Strahlung oder Radiosity.** Sie besteht aus der **Summe der abgestrahlten Energie, der reflektierten Energie und der Energie, die durch die Fläche hindurchtritt.**

# Beispiel



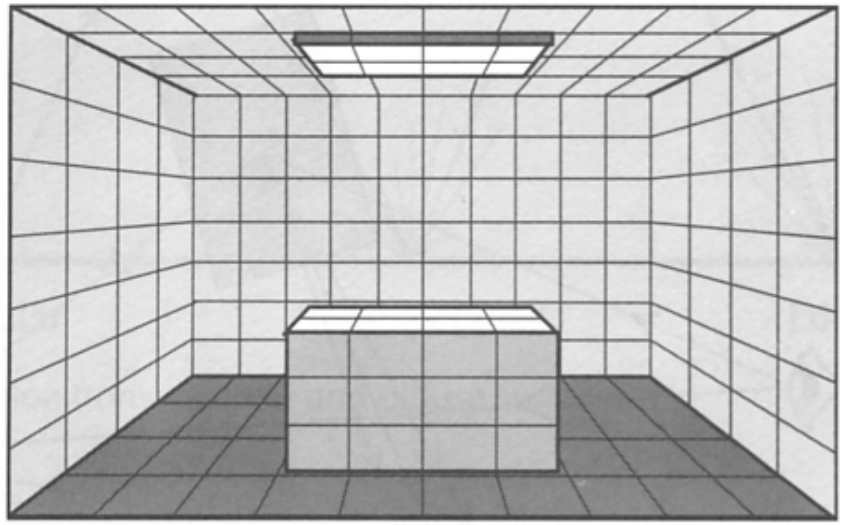
**roter Farbton**

**blauer Farbton**

**Schatten** (Berechnung nicht Teil von Radiosity !)

# Berechnung der Radiosity

**Aufteilung der Szene in Facetten**



**Radiosity der *i*-ten Facette** = **emittierte Energie der *i*-ten Facette** + **reflektierte Energie der *i*-ten Facette**

$$B_i = E_i + R_i \sum_{j=1}^{n} B_j F_{ij}$$

**Radiosity der *i*-ten Facette** = **emittierte Energie der *i*-ten Facette** + **reflektierte Energie der *i*-ten Facette**

$$B_i = E_i + R_i \sum_{j=1}^{n} B_j F_{ij}$$

**reflektierte Energie der *i*-ten Facette** = **Reflexionsindex der *i*-ten Facette** * **reflektierte Energie anderer Facetten auf die *i*-te Facette**

$$R_i \sum_{j=1}^{n} B_j F_{ij} = R_i * \sum_{j=1}^{n} B_j F_{ij}$$

**reflektierte Energie der *j*-ten Facette auf die *i*-te Facette** = **Radiosity der *j*-ten Facette** * **Anteil der von der *j*-ten Facette ausgehenden Energie, die die *i*-te Facette erreicht**

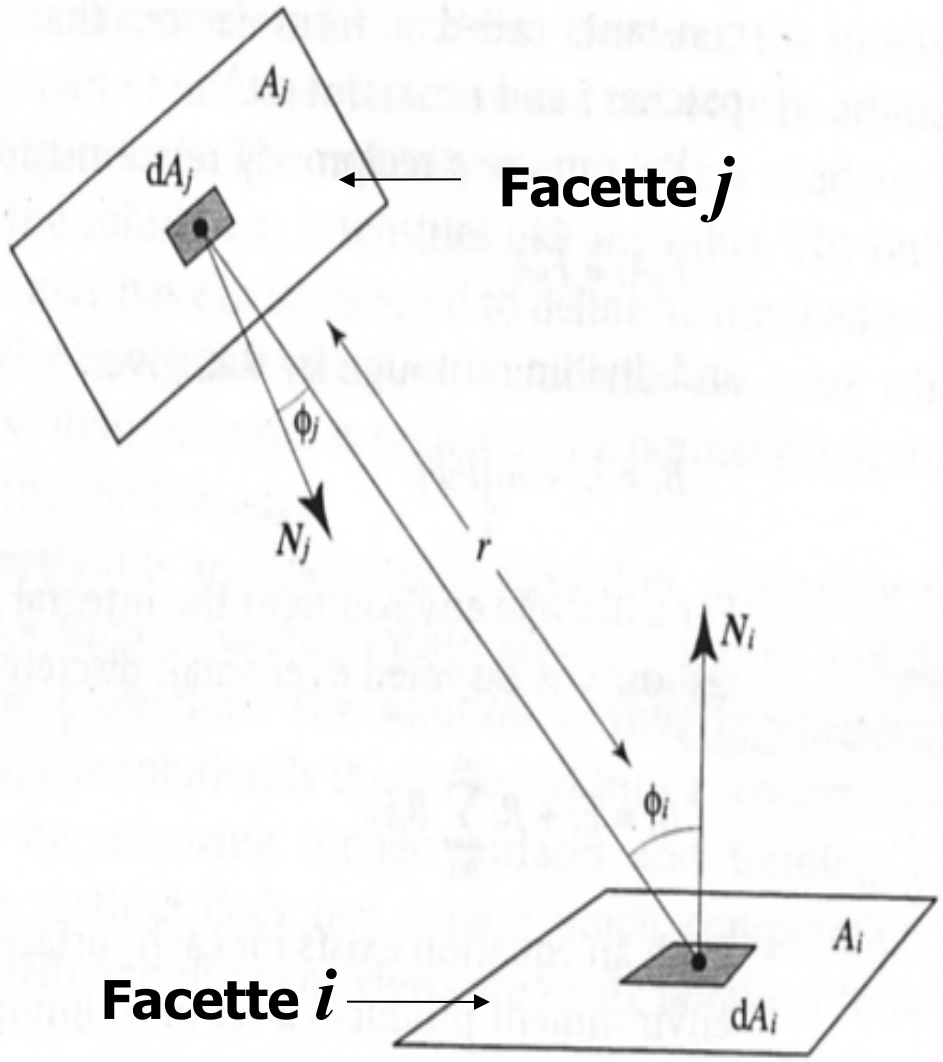
$$B_j F_{ij} = B_j * F_{ij}$$

**Formfaktor**

# Berechnung der Formfaktoren

- **Formfaktor $F_{ij}$:**
  - **Anteil der von der *j*-ten Facette ausgehenden Energie, die die *i*-te Facette erreicht**

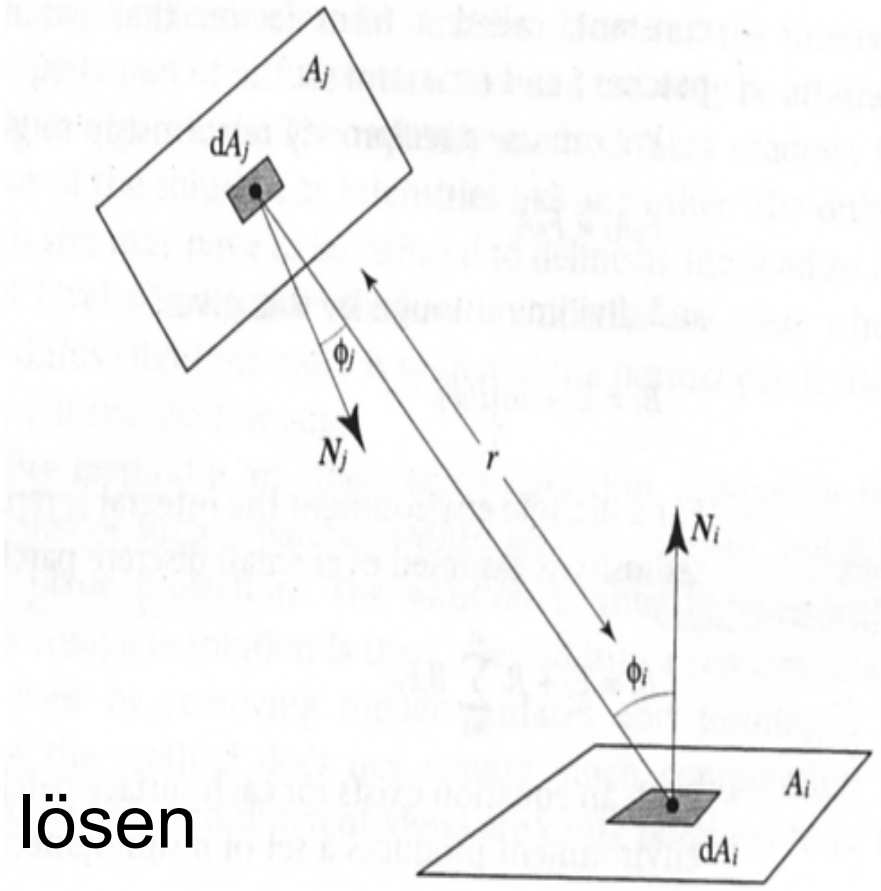
- **Der Formfaktor hängt ab von**
  - **geometrischen Beziehungen zwischen den beiden Facetten**
  - **insbesondere von deren Abstand und Ausrichtung zueinander.**
  - **Es wird nicht berücksichtigt, ob ein Objekt im Weg ist!!!**



# Berechnung der Formfaktoren

- Man kann zeigen, dass für den Formfaktor folgende Gleichung gilt:

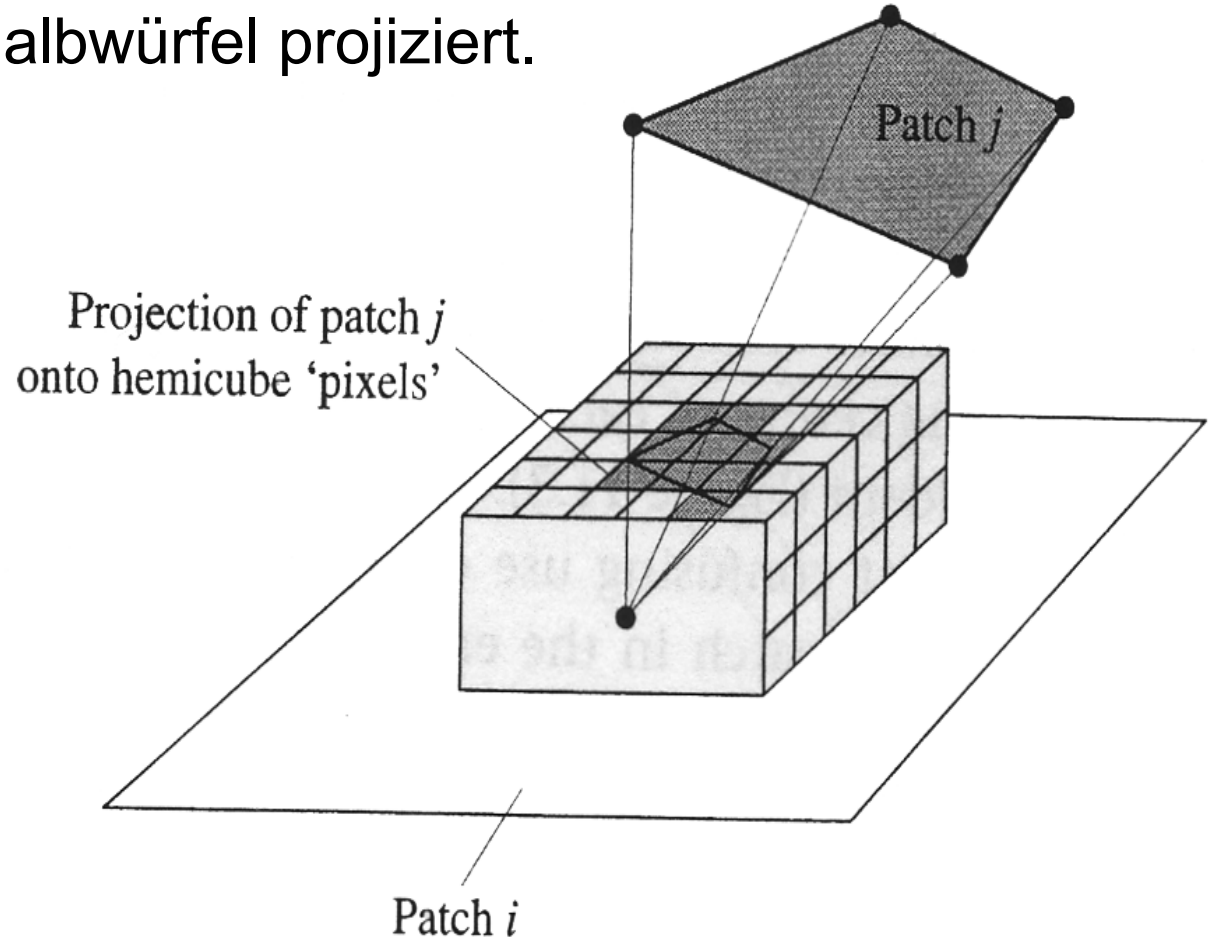
$$F_{ij} = \frac{1}{A_i} \int_{A_i} \int_{A_j} \frac{\cos\theta_i \cos\theta_j}{\pi r^2} dA_j dA_i$$




- Das doppelte Integral ist schwierig zu lösen und wird in der Regel approximiert.

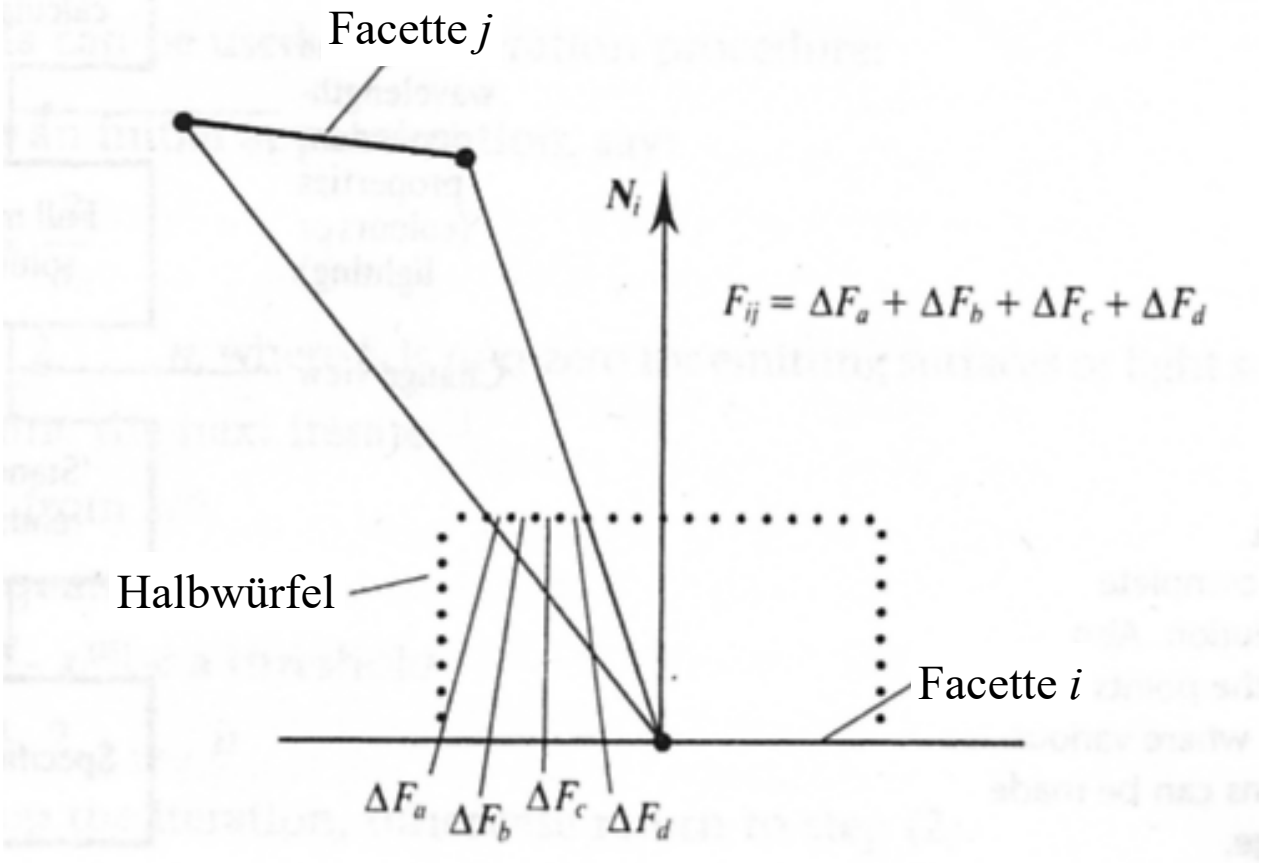
# Approximation mit Halbwürfeln

- Der **Halbwürfel** wird in der Mitte der **Facette *i*** platziert und die Oberfläche des Halbwürfels in kleine Quadrate aufgeteilt.
- Die **Facette *j*** wird auf den Halbwürfel projiziert.



# Approximation mit Halbwürfeln

Identifikation aller Quadrate, auf die die Facette $j$ projiziert wird und Aufsummierung der identifizierten $\Delta$-Formfaktoren (jedes Quadrat hat einen festen Wert) ergibt Formfaktor von Facette $i$ zur Facette $j$



# Beispiel



Weiche Schatten

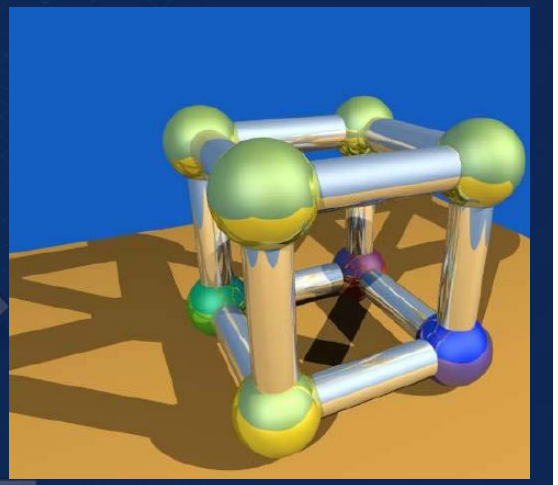
Color Bleeding

Bildquelle: Wikipedia

Von David Bařina, Kamil Dudka, Jakub Filák, Lukáš Hefka (see README) - http://dudka.cz/rrv/files/screenshot/room4-step079-snapshot000.png (RRV - Radiosity Renderer and Visualizer, dudka.cz/rrv), CC BY-SA 3.0, Link

# Vorlesung: Grundlagen der Computergrafik

Kapitel 5: Radiosity und Ray Tracing



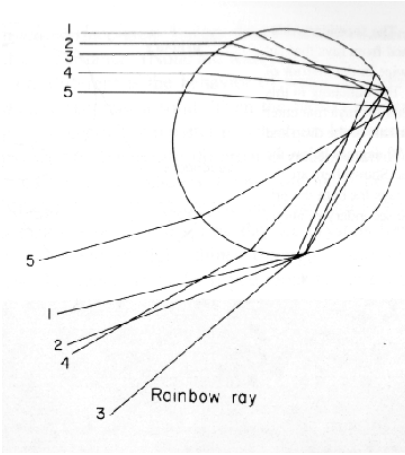
## Kapitel 5.2:

# Ray Tracing

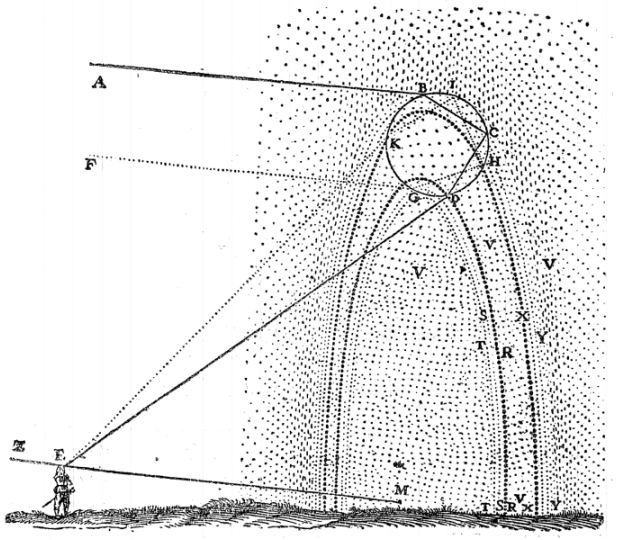
**Vorlesung im Sommersemester 2026**

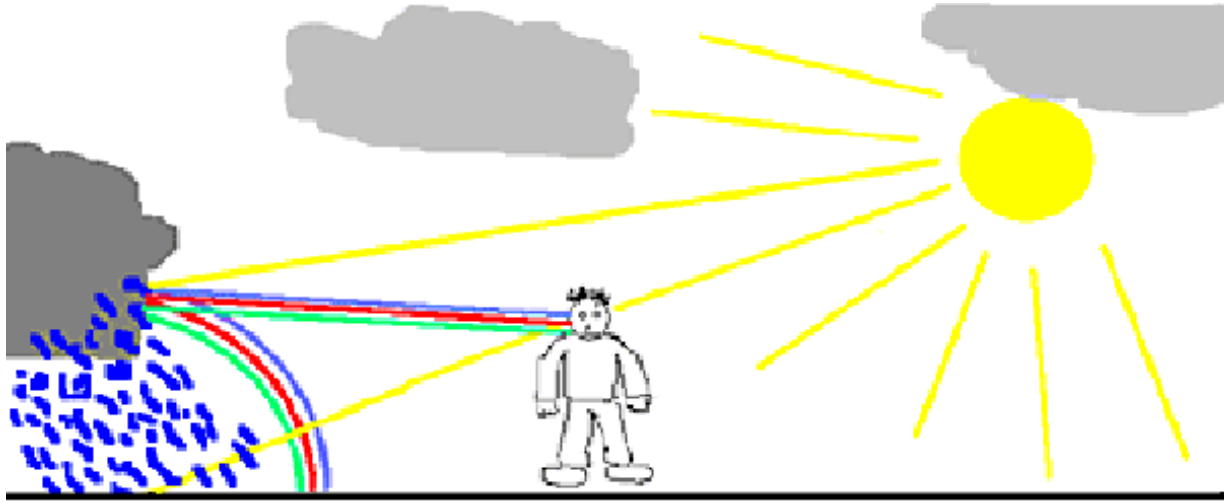
**Prof. Dr. Stephan Diehl, Informatik, Universität Trier**

# Ray Tracing



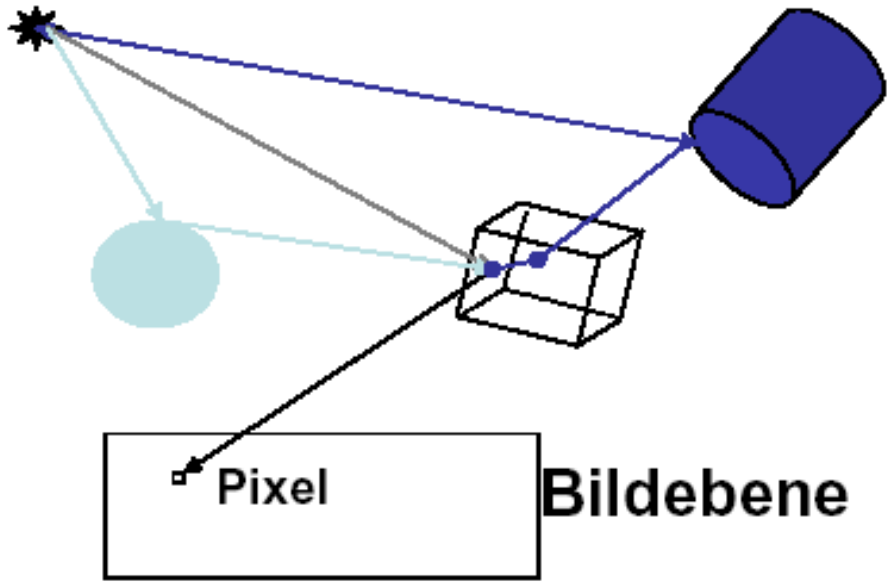

- 1637: Descartes verwendet "ray tracing" zur Erklärung der Entstehung von Regenbogen.
- 1968: Arthur Appel stellt den ersten Algorithmus für Ray Tracing vor.
- 1980: Turner Whitted führt rekursiven Ansatz (Spiegelung, Schatten) ein *("An Improved Illumination Model for Shaded Display"*).





# Ray Tracing (nach Whitted)

- Ein Lichtstrahl reflektiert direkt vom Würfel zum aktuellen Pixel.
- Ein Lichtstrahl reflektiert an der Kugel und dann am Würfel.
- Ein Lichtstrahl reflektiert am Zylinder, per Transmission (zweimalige Brechung) durch den Würfel gelangt er zum aktuellen Pixel.




**Wir unterscheiden zwischen direkten und indirekten Strahlen. In diesem Beispiel ist nur der erste Strahl direkt.**

# Strahlverfolgung

- Alle Strahlen werden **so lange** verfolgt, bis
  - die Strahlenergie unter ein vorgegebenes Kriterium fällt, oder
  - der Strahl die Szene verlässt,
  - eine vorgegebene Rekursionstiefe erreicht ist,
  - oder eine rein-diffuse Fläche getroffen wird.

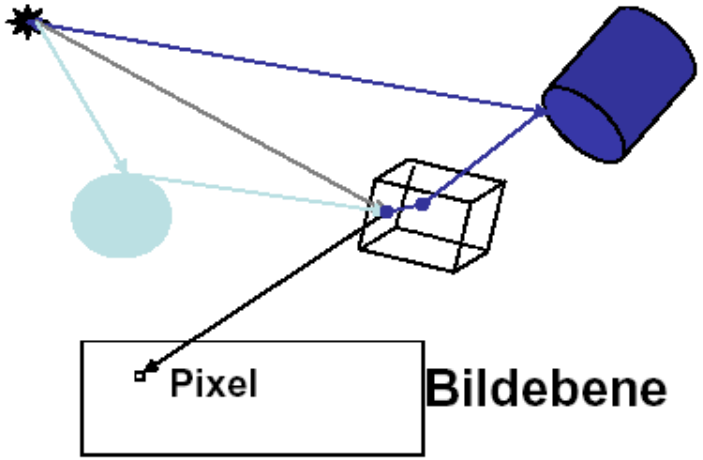
# Farbe

- In jedem getroffenen Punkt ist die Farbe gegeben durch

$$I = I_{lokal} + \mathbf{I}_{\mathbf{Reflektion}} + \mathbf{I}_{\mathbf{Transmission}}$$

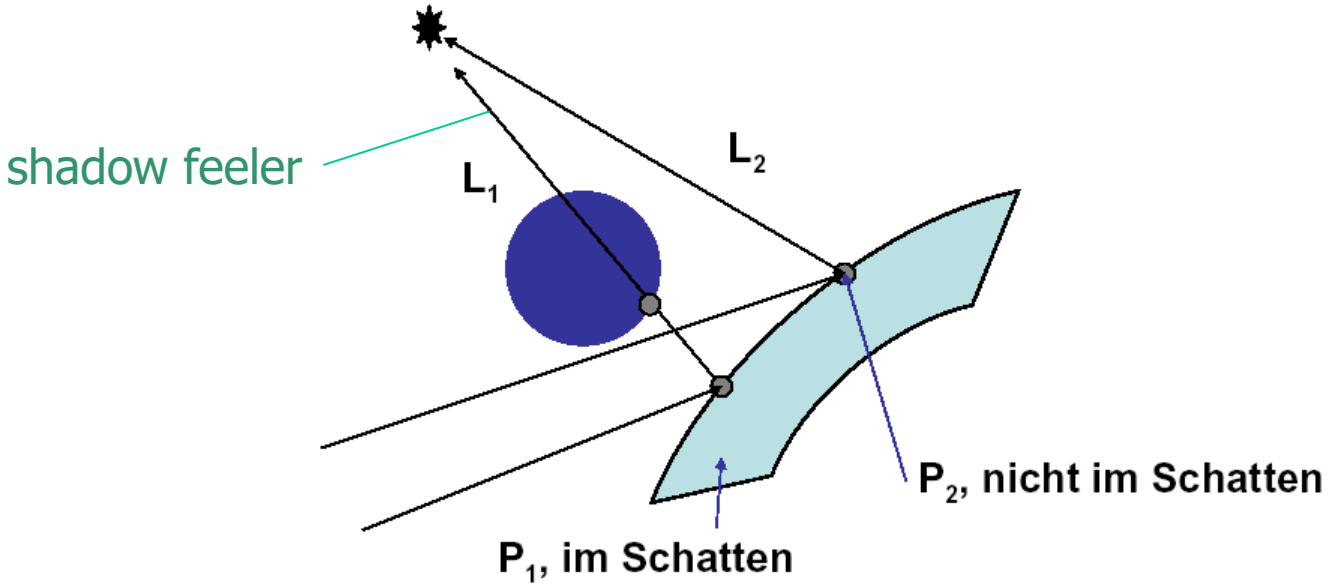
- Der erste $I_{lokal}$ Term wird wie in den lokalen Modellen, z.B. durch Phong, berechnet.

- Für perfekte Spiegel ist $I_{lokal}=0$.



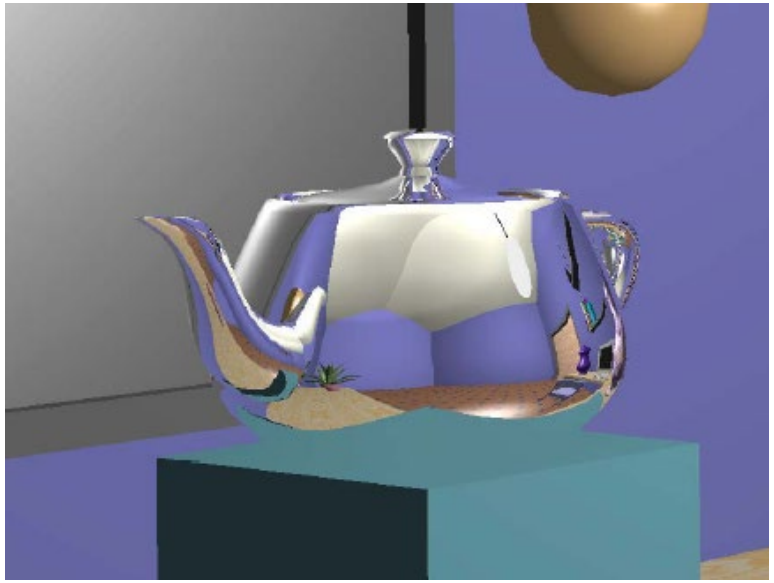
# Schatten

- Shadow feeler = direkter Lichtstrahl zwischen einem Schnittpunkt und einer Lichtquelle
- Trifft er ein anderes Szenenobjekt, dann liegt der Schnittpunkt im Schatten der Lichtquelle, in diesem Fall wird die Lichtquelle nicht bei der Berechnung von $I_{lokal}$ berücksichtigt.

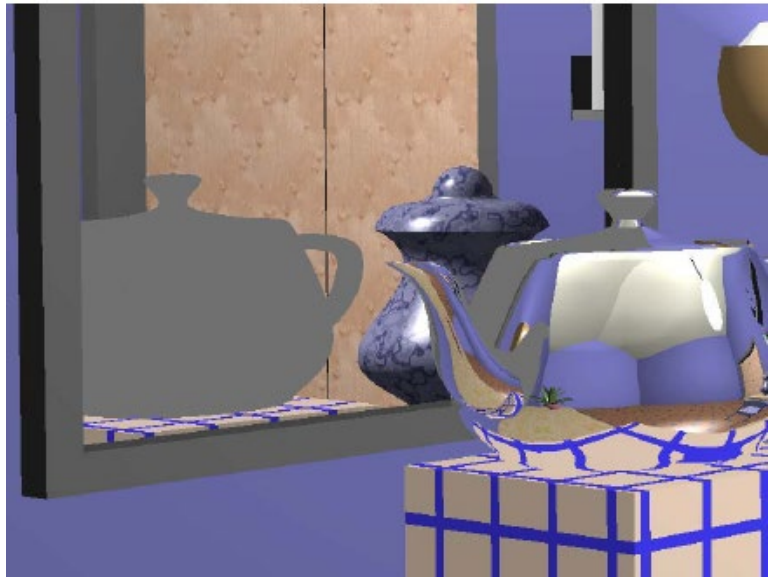



Phong Shading

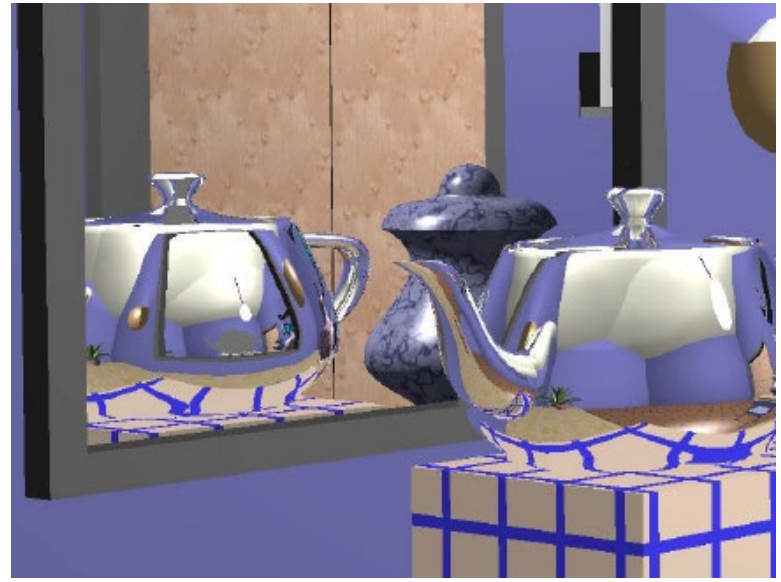

Ray Tracing



Rekursiontiefe 1



Rekursiontiefe 2



Rekursiontiefe 3

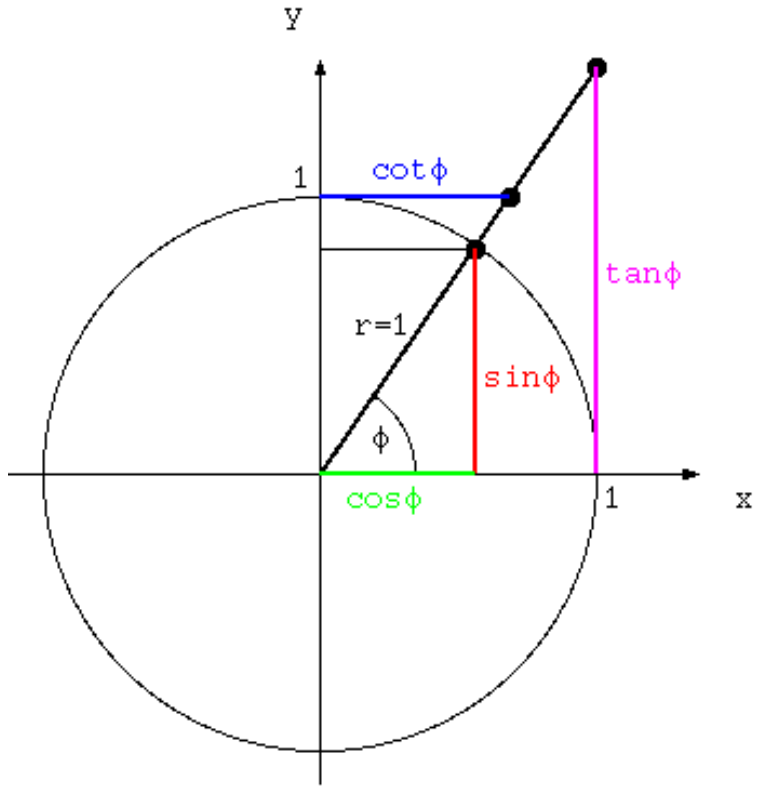
Chapter
5.2.1

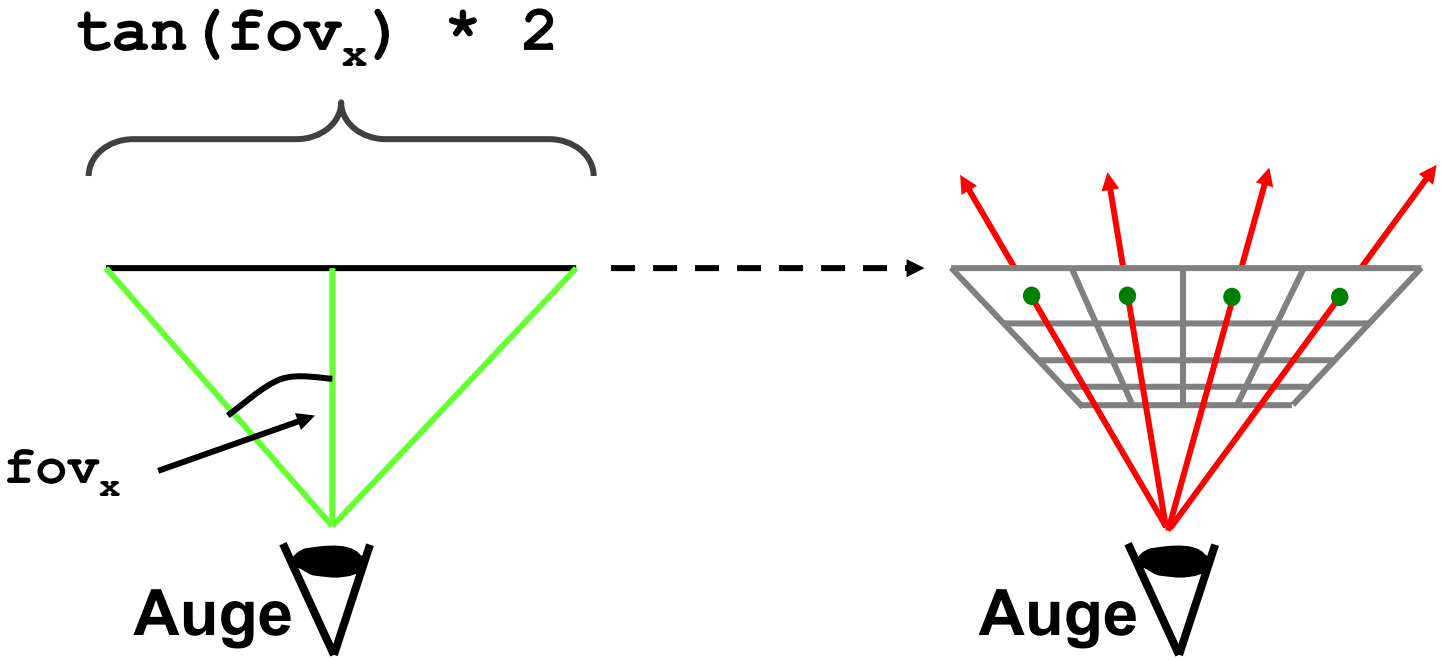
# Strahlerzeugung

# Strahlerzeugung

- Field of View:

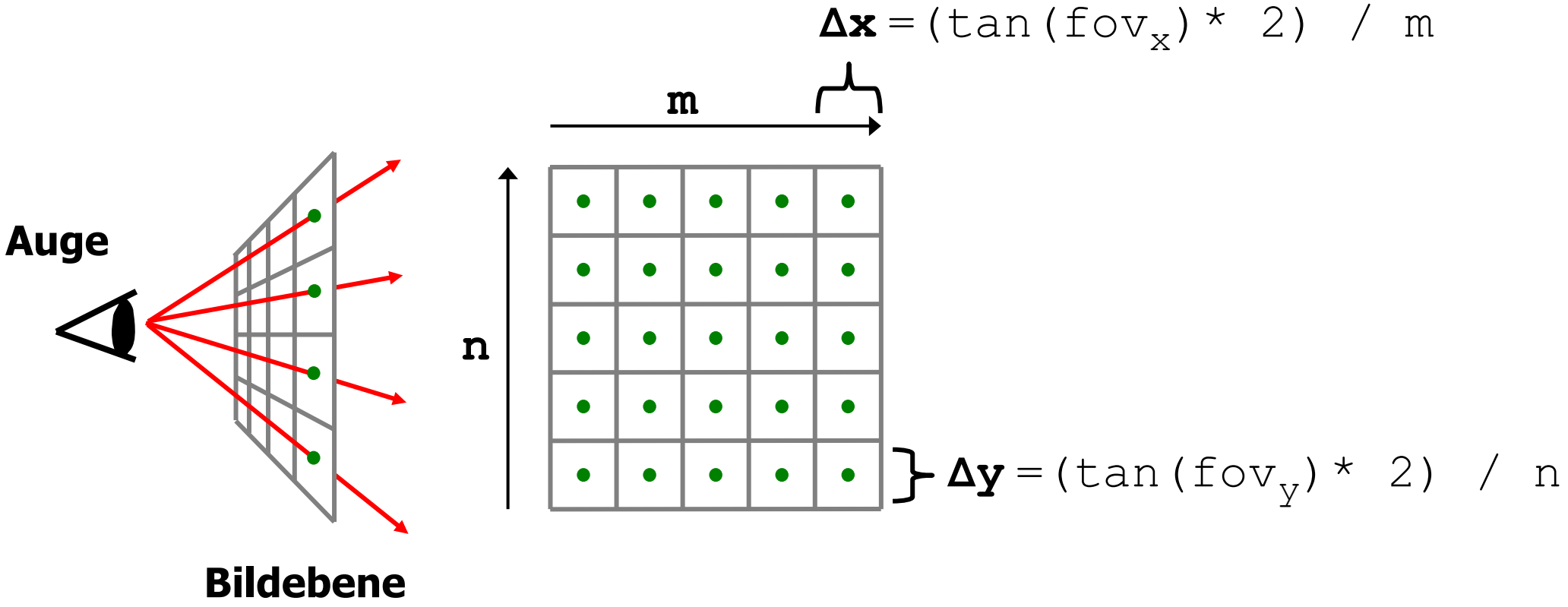
  $fov_x$ = horizontaler Betrachtungswinkel

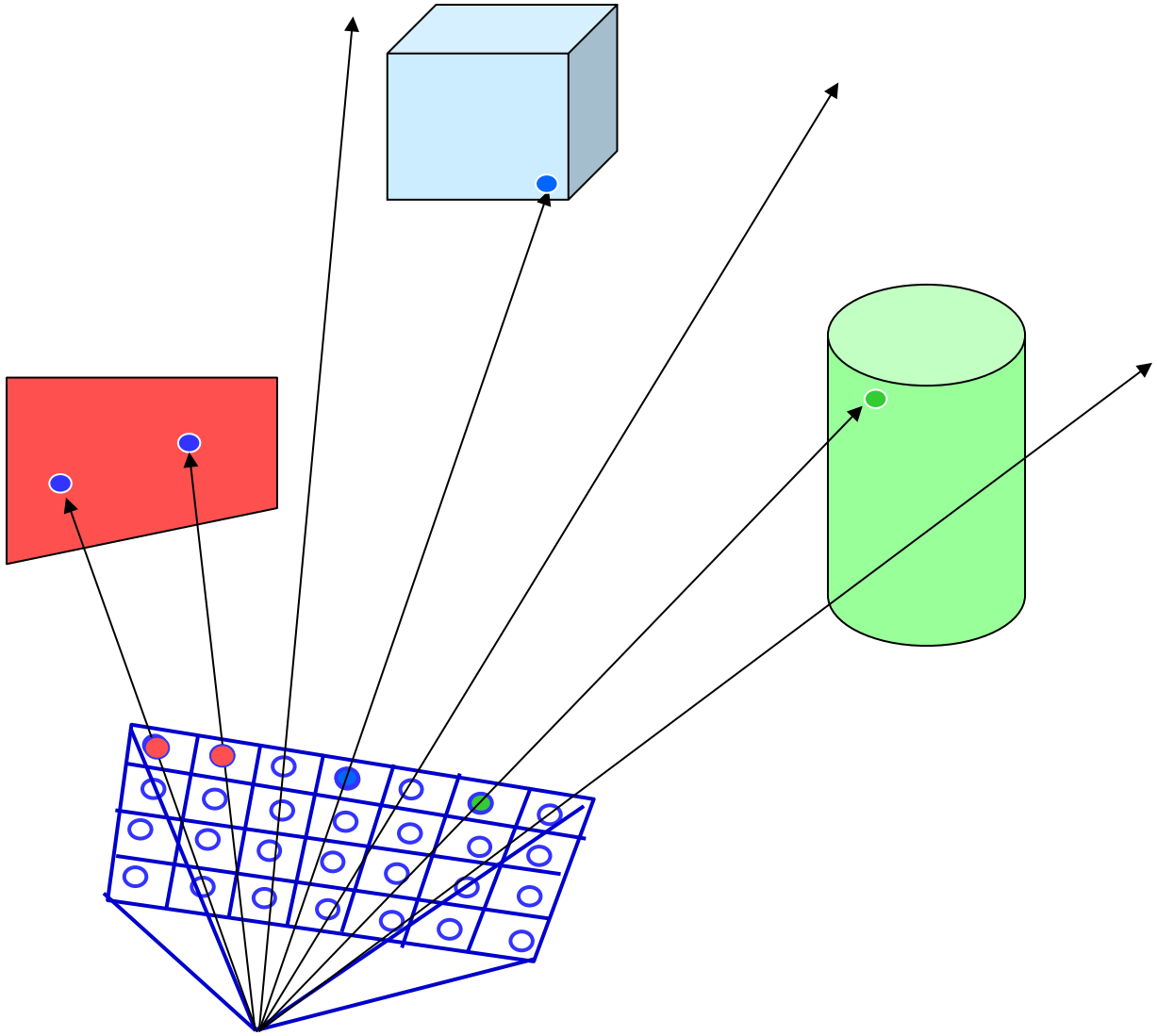


# Strahlerzeugung

- Erzeuge einen Strahl für jeden Pixel des Bildes

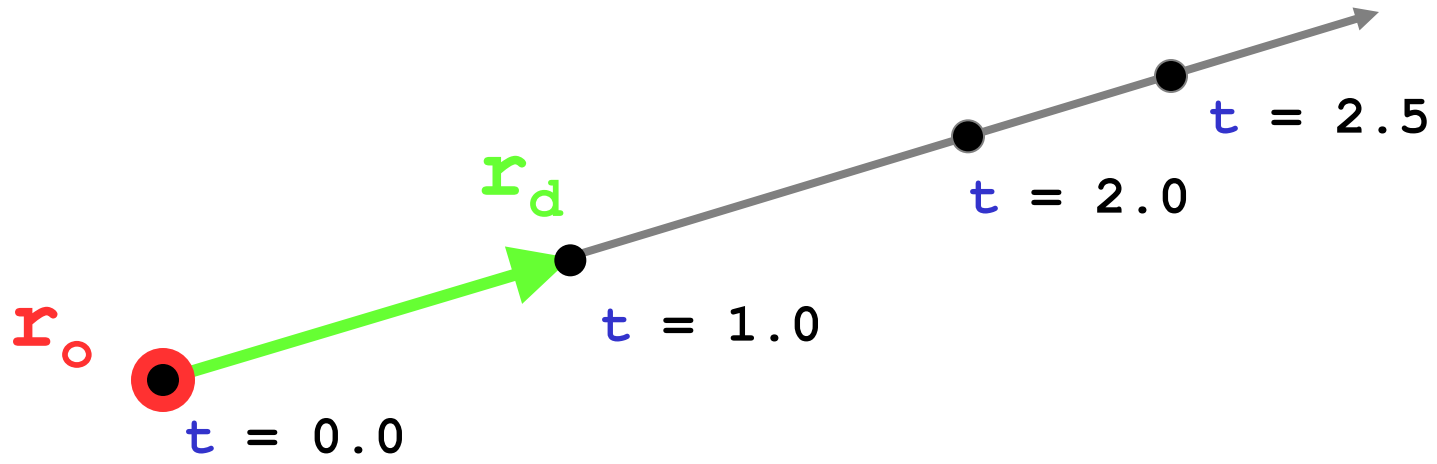




# Parameterdarstellung des Strahls

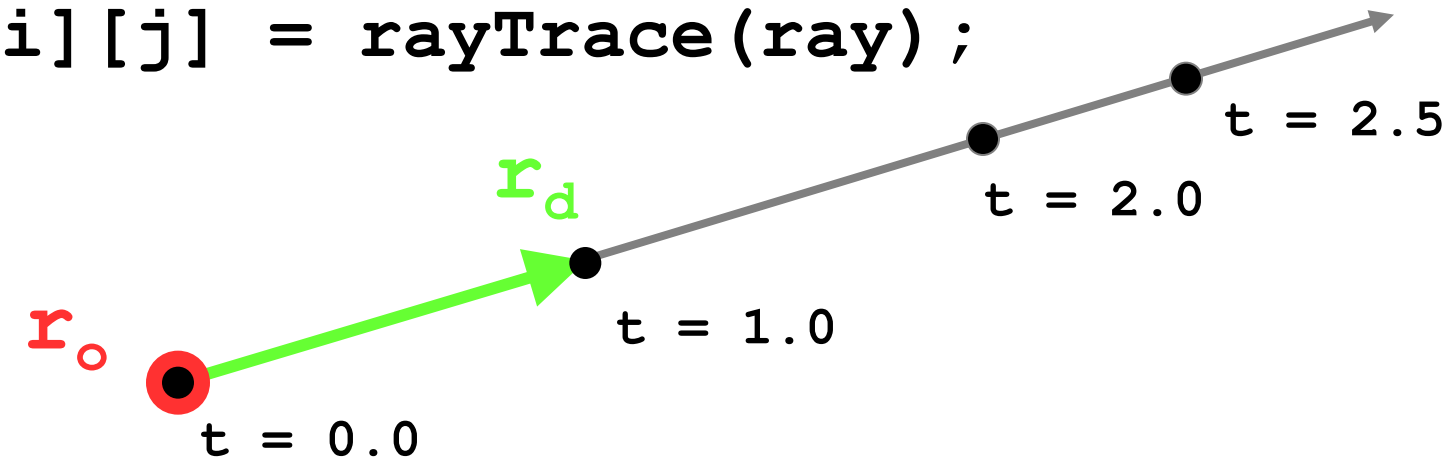
- Der Strahl wird als Funktion eines einzigen Parameters **t** dargestellt:

$$\langle x, y, z \rangle = r_o + t * r_d$$



# Strahlerzeugung

```
renderImage(){
    for each pixel i, j in the image
        ray.setStart(0, 0, 0);    // ro
        ray.setDir  ((.5 + i) * tan(fovx)* 2 / m,
                     (.5 + j) * tan(fovy)* 2 / n,
                      1.0);       // rd
        ray.normalize();
        image[i][j] = rayTrace(ray);
}
```



Chapter
5.2.2

# Schnittpunktberechnung

# Implementierung eines Raytracers

- Wir müssen Schnittberechnungen zwischen geometrischen Objekten und Geraden (Strahlen) durchführen.

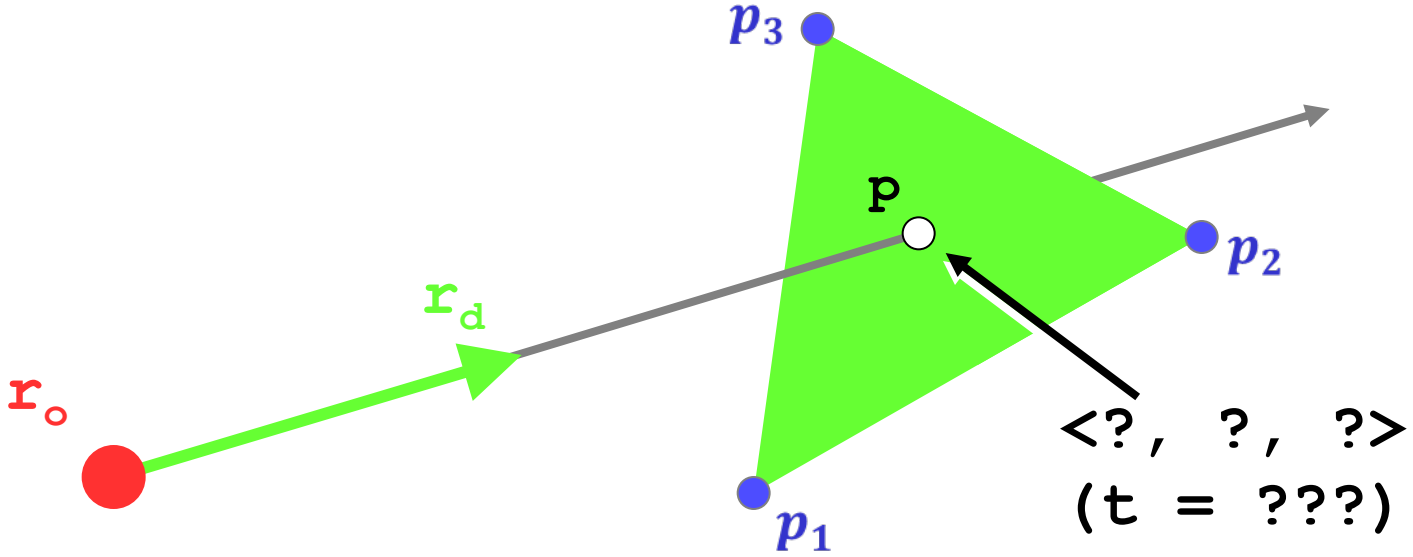
- Whitted schätzt: **95 % der Zeit** eines rekursiven Ray-Tracers wird mit der **Schnittpunktberechnung** verbracht!



*Quelle: T.* Whitted. An improved illumination model for shading display. Communications of the ACM, 23(6):343–349, 1980

# Schnitt mit einem Dreieck

- Geben Sie hier eine Formel ein.Zuerst: Berechne Schnittpunkt **p** des Strahls mit dem Dreieck
- Anschließend werden Beleuchtung, reflektierte Strahlen und Schatten dieses Punktes berechnet.




$$\vec{n'} = (\vec{p_3} - \vec{p_1}) \times (\vec{p_2} - \vec{p_1})$$

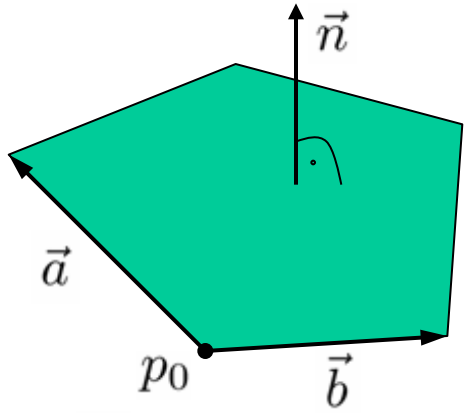
$$\vec{n} = \frac{\left|\vec{n'}\right|}{\left|\vec{n'}\right|}$$

# Polygone, Ebenen, Flächennormale

Inneres Produkt: $\vec{a} \cdot \vec{b} = \sum_{i=1}^{n} a_i b_i$
Norm: $|\vec{a}| = \sqrt{\vec{a} \cdot \vec{a}}$
Winkel zwischen Vektoren: $\vec{a} \cdot \vec{b} = |\vec{a}||\vec{b}| cos\ \alpha$

Vektorprodukt (hier nur 3D): $\vec{a} \times \vec{b} = \begin{pmatrix} a_y b_z - a_z b_y \\ a_z b_x - a_x b_z \\ a_x b_y - a_y b_x \end{pmatrix}$

$\vec{a} \times \vec{b}$ steht senkrecht auf der von $\vec{a}$ und $\vec{b}$ aufgespannten Ebene.

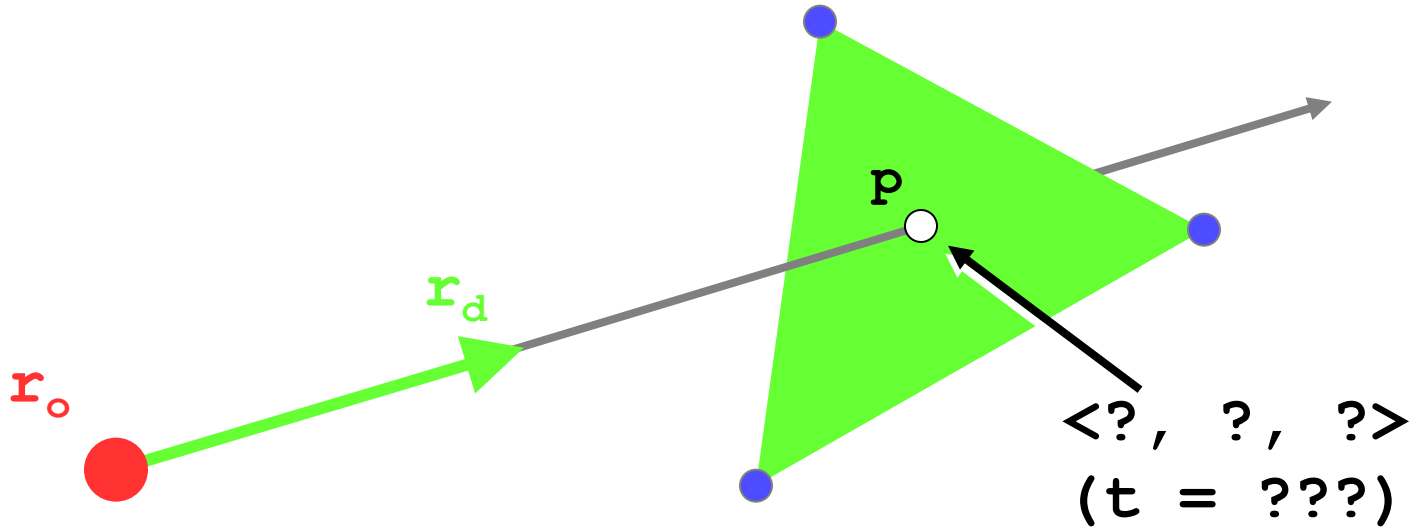



**$\vec{a}$, $\vec{b}$ und $\vec{n}$ bilden ein rechthändiges System**

Seien $\vec{a}$ und $\vec{b}$ benachbarte gerichtete Kanten (siehe Abbildung):
Normale: $\vec{n} = \vec{b} \times \vec{a}$ (im Gegenuhrzeigersinn, zeigt nach "Außen")

Ebenengleichung: Alle Punkte $\vec{P}$ der Ebene erfüllen die Gleichung $\vec{n} \cdot \vec{p} = d$

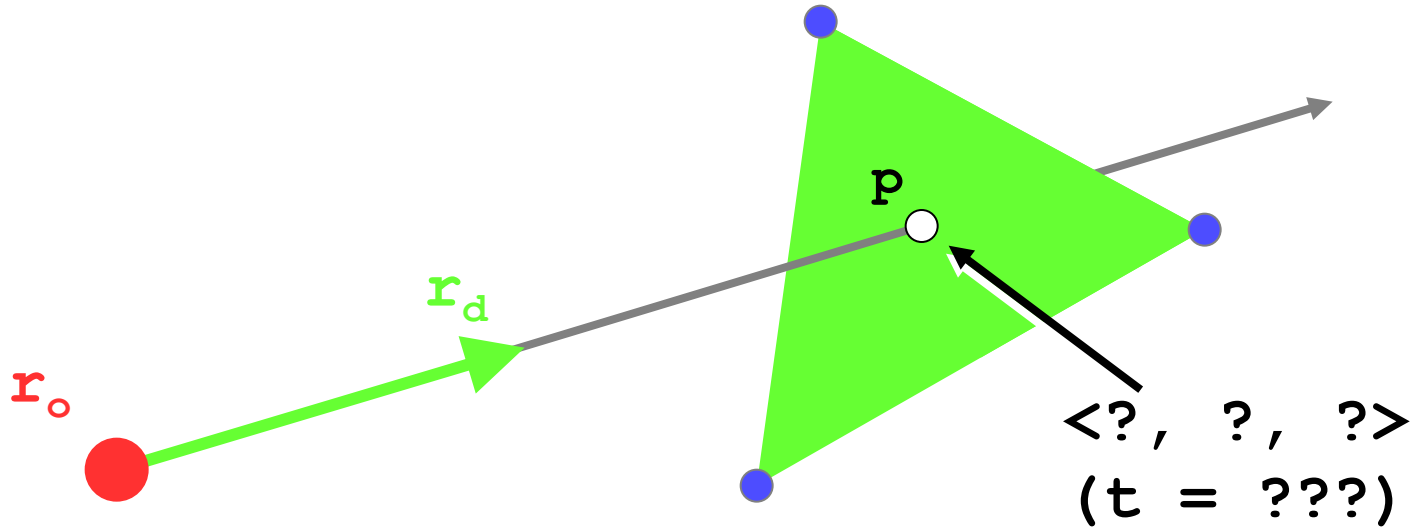
Ist ein Punkt $p_0$, z.B. ein Eckpunkt des Polygons, gegeben, so kann man $d$ direkt berechnen.


p
$r_d$
$r_o$
<?, ?, ?>
(t = ???)

$$p = r_o + t\, r_d$$
$$n \cdot p = d$$

$$n \cdot (r_o + t\, r_d) = d$$
$$n \cdot r_o + t\, n \cdot r_d = d$$

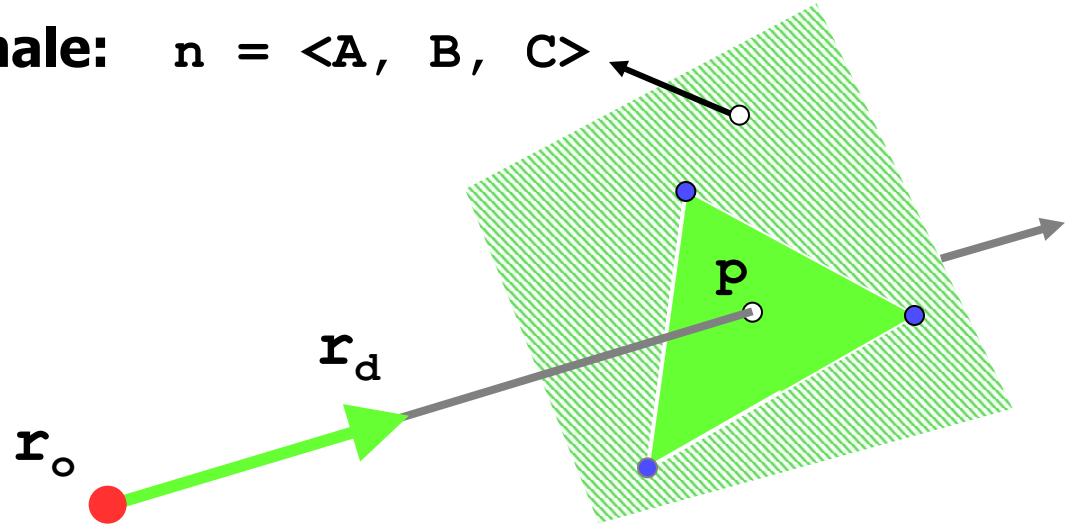
$$t = \frac{d - n \cdot r_o}{n \cdot r_d}$$



# Schnitt mit einem Dreieck

**Erster Schritt:** Schnitt mit der Ebene in der das Dreieck liegt ( $Ax + By + Cz + D = 0$ )

**Flächennormale:** $n = \langle A, B, C \rangle$




**D = Abstand vom Ursprung**

**$\hat{v}$ normalisierter Vektor v**

$$t = (D - \hat{n} \cdot r_o) \; / \; (\hat{n} \cdot \hat{r}_d)$$

Für dieses **t** schneidet der Strahl die Ebene im Punkt **p**.

# Schnitt mit einem Dreieck

- **Zweiter Schritt :** Überprüfe Lage des Schnittpunktes mit Hilfe der Eckpunkte

**Berechne barizentrische Koordinaten $\alpha$, $\beta$:**

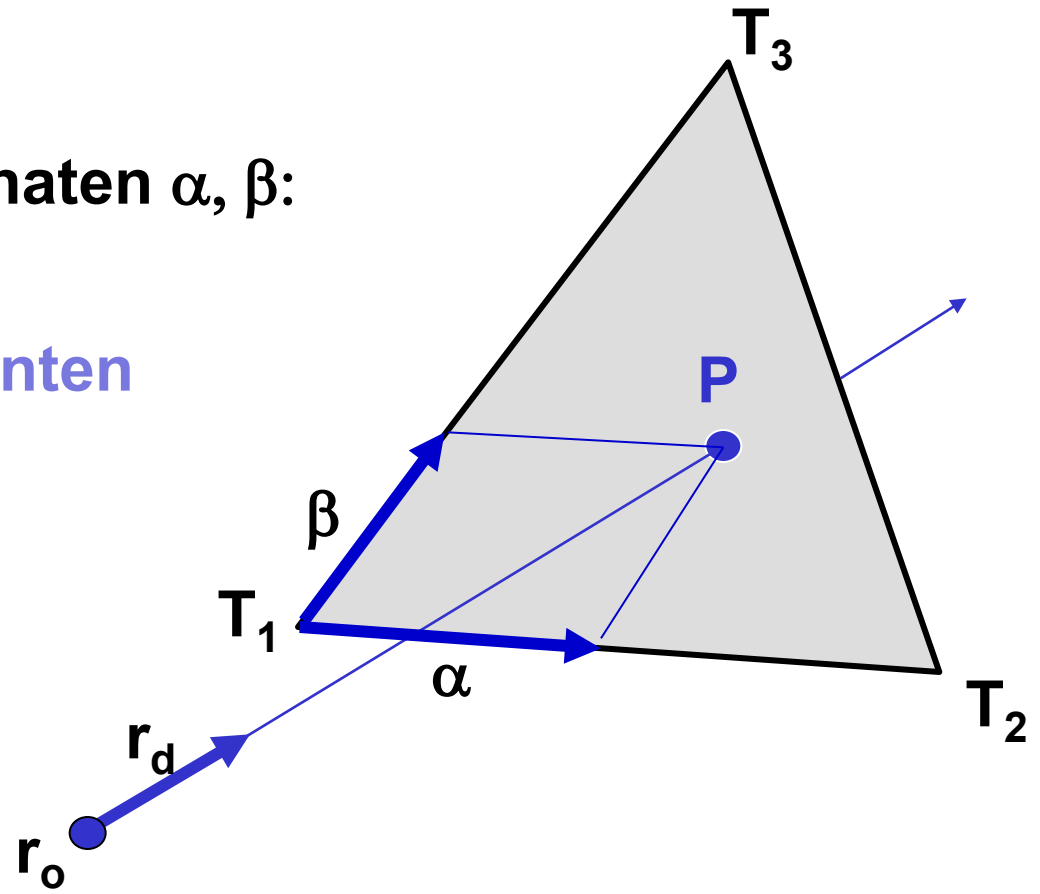
$$P = T_1 + \alpha (T_2 - T_1) + \beta (T_3 - T_1)$$

Drei Gleichungen mit 2 Unbekannten

**Der Punkt liegt im Dreieck, falls**

$$0 \leq \alpha \leq 1 \text{ and } 0 \leq \beta \leq 1$$

$$\alpha + \beta \leq 1$$



# Berechnung der Schnittpunkte

- Prüfe alle Dreiecke und liefere den Schnittpunkt mit dem nächsten Dreieck zurück

```
hitObject(ray) {
   for each triangle in scene
      does ray intersect triangle?
      if (intersected and was closer)
         save that intersection
   if (intersected)
      return intersection point and normal
}
```

Ein schnelles Verfahren zur Schnittpunktberechnung wird z.B. im Buch „Realtime Rendering“ von Tomas Akenine-Möller und Eric Haines angegeben.

Chapter
5.2.3

# Pseudo-Code: Rekursives Ray Tracing

# Rekursives Ray-Tracing

```
rayTrace(ray) {
    (p, n, triangle) = hitObject(ray)
    color = object color;
    if (object is light)
        return(color);
    else
        return( lighting(p, n, color) );
}
```

# Rekursives Ray-Tracing

- Berechnung der Oberflächenfarbe

```
lighting(point) {
    color = ambient color;
    for each light
        if (!hitObject(shadow ray))
            color += lightcolor * dot(shadow ray, n);
    color += rayTrace(reflection) *
        pow(dot(reflection, ray), shininess);
    return( color );
}
```

$$I = I_a * k_a + \sum_{ls \in LightSources} f_{at}^{li} * (I_d^{li} + I_s^{li})$$

# Strahlverfolgung

- Alle Strahlen werden **so lange** verfolgt, bis
  - die Strahlenergie unter ein vorgegebenes Kriterium fällt, oder
  - der Strahl die Szene verlässt,
  - eine vorgegebene Rekursionstiefe erreicht ist,
  - oder eine rein-diffuse Fläche getroffen wird.

# Rekursives Ray-Tracing

```
rayTrace(ray) {
    (p, n, triangle) = hitObject(ray)
    color = object color;
    if (object is light)
        return(color);
    else
        return(lighting(p, n, color));
}
```

## Strahlverfolgung

Alle Strahlen werden **so lange** verfolgt, bis

- die Strahlenergie unter ein vorgegebenes Kriterium fällt, oder
- der Strahl die Szene verlässt,
- eine vorgegebene **Rekursionstiefe** erreicht ist,
- oder eine rein-diffuse Fläche getroffen wird.

```
lighting(point) {
    color = ambient color;
    for each light
        if (!hitObject(shadow ray))
            color += lightcolor * dot(shadow ray, n);
    color += rayTrace(reflection) *
        pow(dot(reflection, ray), shininess);
    return(color);
}
```

# Rekursives Ray-Tracing

- Hauptprogramm

```
main() {
    triangles = readTriangles();
    image = renderImage(triangles);
    writeImage(image);
}
```

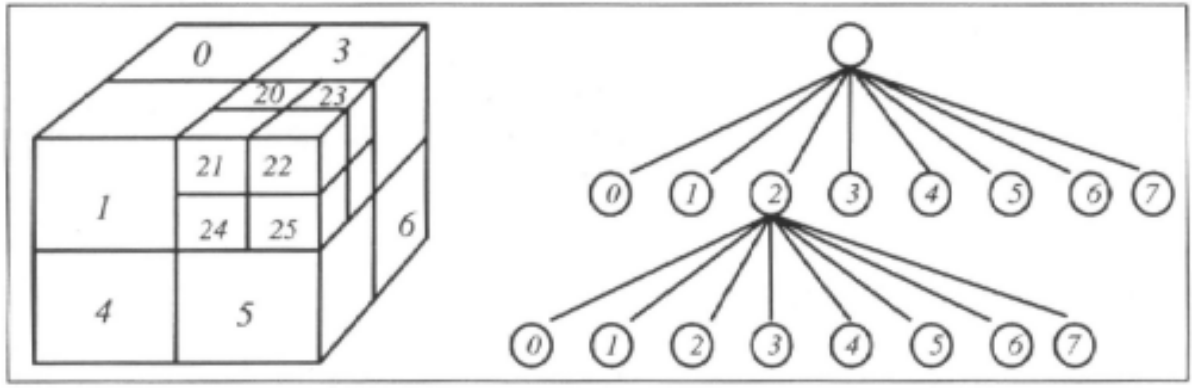
# Rechenaufwand bei Raytracing

- 1280x1024 Bild mit einem Strahl pro Pixel
- 1000 Dreiecke
- Rekursionstiefe 3

=> 3.932.160.000 Tests (Schnittberechnungen)

bei 100.000 Tests/Sekunde -> 10 Tage!

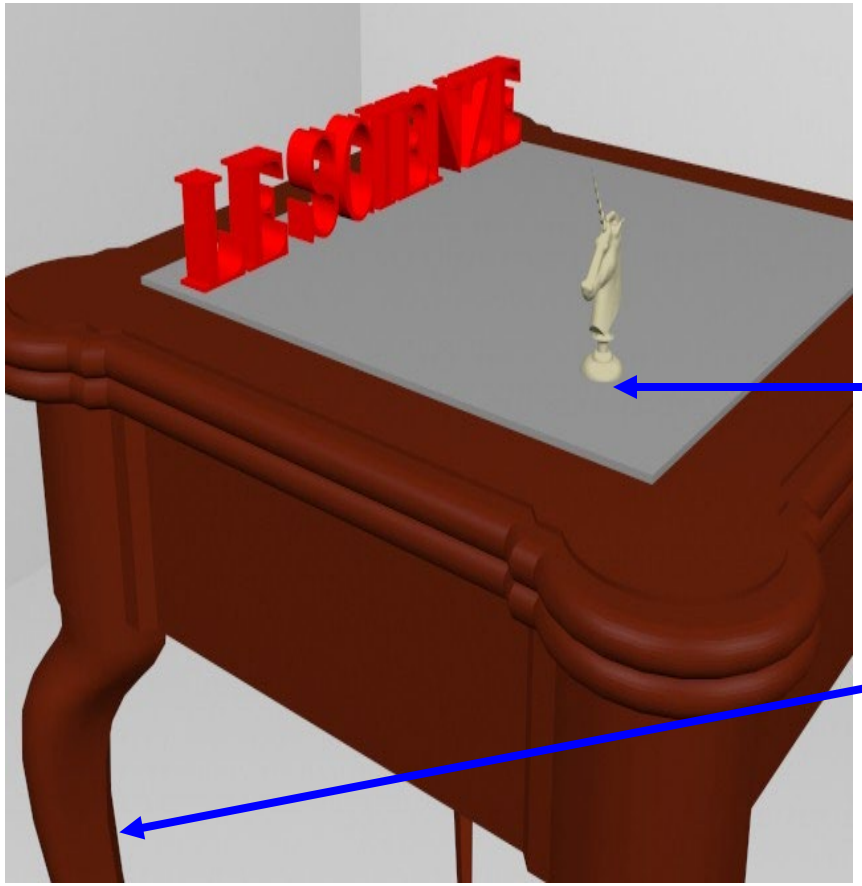
→ Beschleunigung durch räumliche Partitionierung, z.B. Octrees



Chapter 5.2.4

# Vergleich mit anderen Techniken

# Vergleich der Techniken: Shading

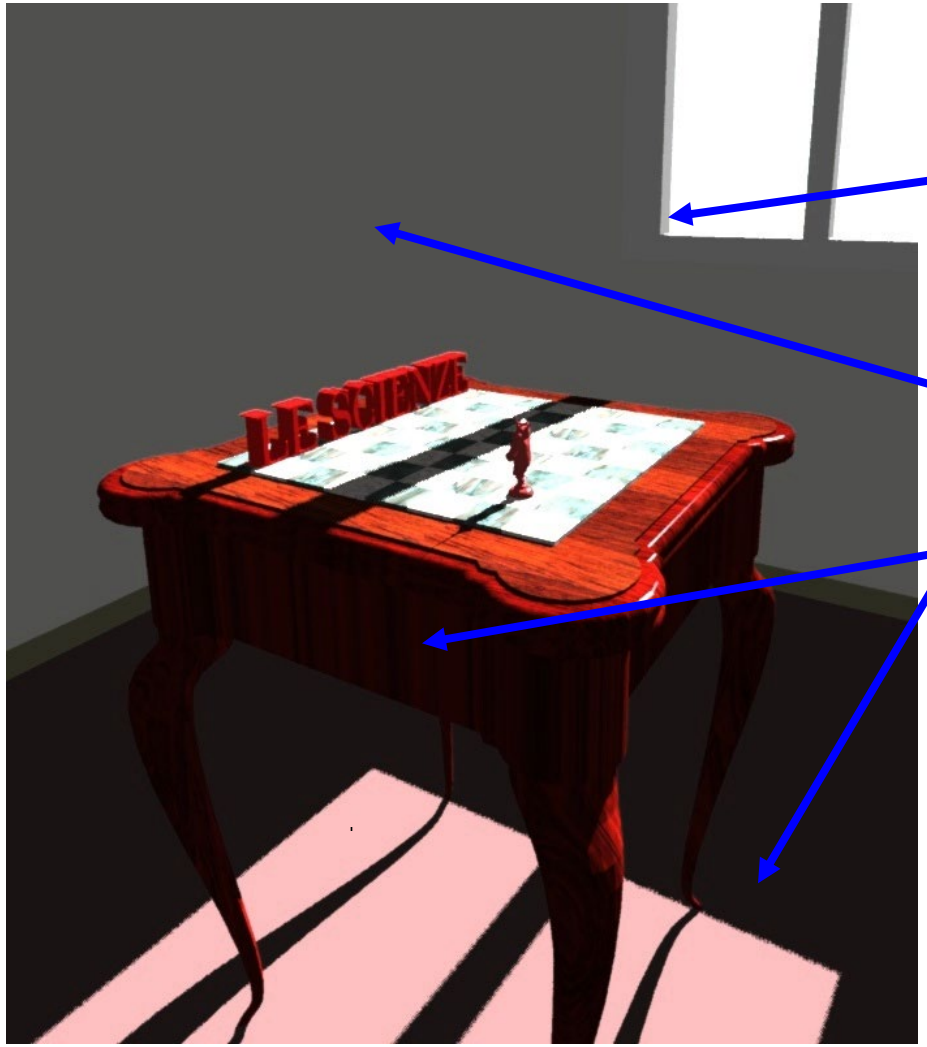


**Szene sieht wie aus Plastik aus**

**keine Schatten**

**keine Interaktionen (indirektes Licht) zwischen den Objekten (Tisch, Wand, Boden)**

# Vergleich der Techniken: Ray Tracing



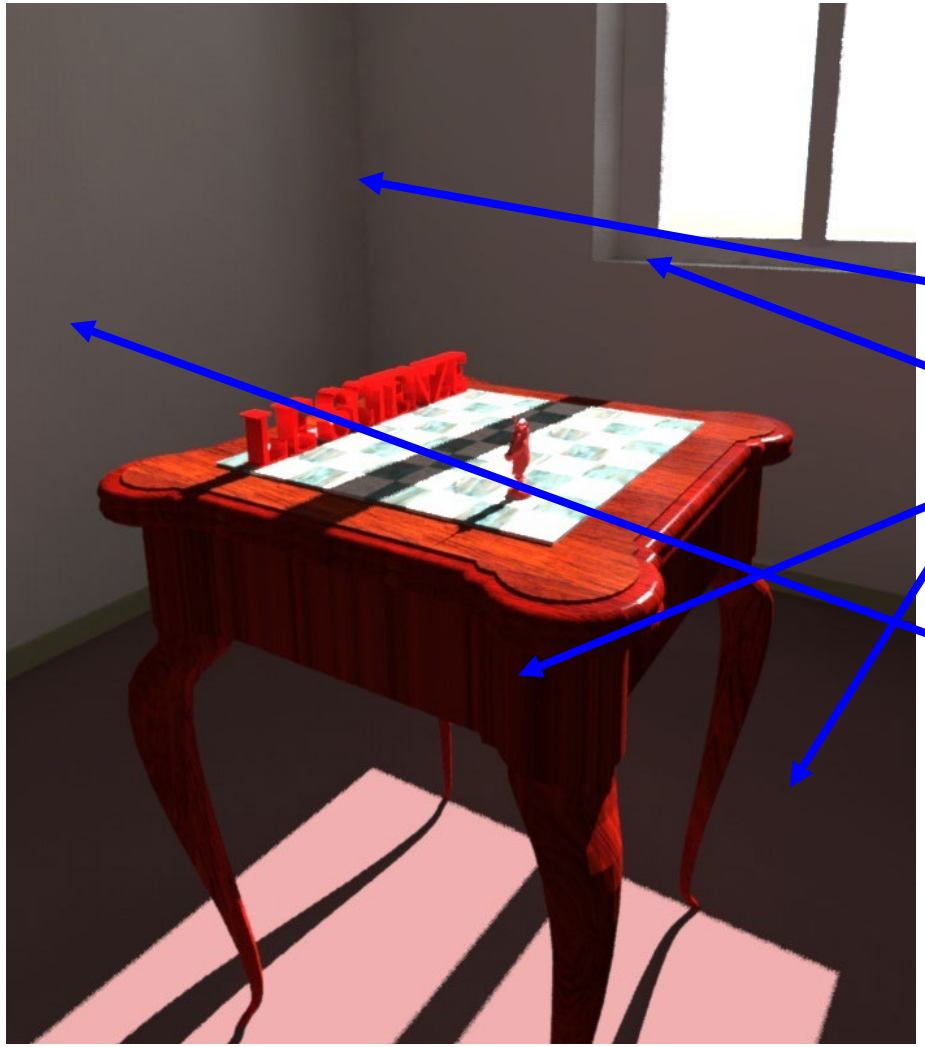
**Szene sieht nicht sehr realistisch aus**

**Ist das Fenster bündig mit der Wand ?**

**Wo ist die Ecke des Raumes ?**

**Sind der Teppich und das Holz in Wirklichkeit so dunkel ?**

# Vergleich der Techniken: Raytracing + Radiosity



**Indirektes Licht verbessert Realismus**

**Der Raum hat eine Ecke**

**Das Fenster hat eine Tiefe**

**Der Teppich und das Holz sind heller**

**Die Wände haben eine rötlichen Farbton**



# Vergleich



**Ray-Tracing**

- image space algorithm (berechnet Rasterbilder)
- abhängig von Kameraposition
- erzeugt harte Schatten (Punktlicht/Richtungslicht)
- Umgebungslicht angenähert (konstant)
- Effekte:
  - Spiegelungen
  - Brechungen
- zeitaufwendig

**Radiosity**

- object space algorithm (berechnet Intensitäten auf Oberflächen)
- unabhängig von Kameraposition
- erzeugt weiche Schatten (Flächenlichtquellen)
- Umgebungslicht korrekt aufgrund diffuser Lichtausbreitung
- Effekte:
  - color bleeding
  - weiche Schatten
- effizient (approximatives Verfahren mit Hemicubes)

# Aufgabe

